ORBIT v1.0

Online Remote Binary Interaction Technology

Comprehensive Technical Specification & Implementation Guide

Version 1.0

Date: June 2026

Table of Contents

Executive Summary

ORBIT v1.0 is a production-grade remote desktop platform designed to provide secure, low-latency remote access to Windows computers through a Chrome Extension. This document serves as the complete technical specification and implementation roadmap for building a professional-quality remote computing platform from the ground up.

Key Focus Areas:

Speed & Low Latency: <40ms on LAN networks

Security: TLS 1.3, DTLS, AES-256-GCM encryption

Resource Efficiency: <100MB RAM idle, <2% CPU usage

Reliability: Production-ready architecture with comprehensive error handling

Scalability: Clean architecture supporting future feature modules

Project Overview

Vision

Build a rock-solid foundation for remote desktop technology that prioritizes stability, security, and performance. While v1.0 focuses on core functionality (screen viewing, mouse control, keyboard input), the architecture is designed to scale into a complete ecosystem supporting cloud PCs, marketplace, gaming optimization, and enterprise features in future versions.

Scope - v1.0 Features

Included:

Remote screen viewing (30-60 FPS)
Mouse control (move, click, double-click, scroll, drag)
Keyboard input with Unicode & shortcut support
Text clipboard synchronization
User authentication (Email, Password, Google OAuth)
Session management with device authorization
Explicitly NOT included in v1.0:
File transfer
Mobile clients
Voice/video chat
Multi-monitor support
Cloud PC services
Target Platforms
Implementation Roadmap
Phase 1: Foundation (Weeks 1-2)
Backend Infrastructure Setup
Project initialization with Rust & Axum
PostgreSQL & Redis database setup
JWT authentication framework
Desktop Agent Foundation
Rust project structure
Windows API bindings (Screen capture, input simulation)
Phase 2: Core Features (Weeks 3-5)
Screen Capture & Encoding
DXGI screen capture implementation
H.264 hardware encoding (Intel Quick Sync, NVIDIA NVENC, AMD AMF)
FPS adjustment algorithm
Input Control
Mouse control (move, click, scroll)
Keyboard input with modifier key support
WebRTC Implementation
WebRTC peer connection setup
STUN/TURN server configuration
Phase 3: Chrome Extension & UI (Weeks 6-8)
Extension Development
Manifest V3 setup with TypeScript
WebRTC video rendering
UI/UX Development

Dashboard with React & Tailwind CSS

Session management UI

Connection status & controls

Phase 4: Security & Testing (Weeks 9-10)

Security Implementation

End-to-end encryption (TLS 1.3, DTLS)

Rate limiting & DDoS protection

Testing & QA

Unit tests for all critical components

Integration testing

Performance benchmarking

Architecture Overview

High-Level Architecture

ORBIT consists of three primary components working together in a distributed architecture:

Chrome Extension (Client)

User authentication interface • Session management • WebRTC peer connection • Remote screen rendering • Input event handling

Desktop Agent (Host)

Screen capture (DXGI) • Video encoding (H.264) • Input simulation (mouse, keyboard) • Clipboard sync • System monitoring

Backend Server

Authentication & authorization • Session management • WebRTC signaling • TURN/STUN services • Logging & monitoring • Device registration

Communication Flow

1. Client initiates login via Chrome Extension → Backend validates credentials → Returns JWT token

2. Client requests session with host device ID → Backend retrieves device info & returns TURN/STUN servers

3. WebRTC signaling via backend WebSocket → ICE candidates exchanged between client & agent

4. P2P connection established → Screen data streamed via WebRTC data channels

5. Input commands sent from client to agent via WebRTC → Agent processes and applies to system

Component Responsibilities

Folder Structure & Project Layout

Project Root Structure

orbit/ ├── backend/ # Rust backend server ├── desktop-agent/ # Rust Windows desktop agent ├── chrome-extension/ # TypeScript Chrome extension ├── docs/ # Documentation ├── docker/ # Docker configuration ├── scripts/ # Build & deployment scripts ├── tests/ # Integration tests ├── .github/workflows/ # CI/CD pipelines ├── Cargo.workspace.toml # Rust workspace config ├── docker-compose.yml # Local

development └── README.md

Backend Structure (Rust + Axum)

backend/ ├── src/ │ ├── main.rs # Application entry point │ ├── config/ │ │ ├── mod.rs │ │ ├── database.rs # PostgreSQL connection pool │ │ ├── redis.rs # Redis client setup │ │ ├── secrets.rs # Environment & secrets │ │ └── server.rs # Server config │ ├── models/ │ │ ├── mod.rs │ │ ├── user.rs # User data model │ │ ├── device.rs # Device registration │ │ ├── session.rs # Session entity │ │ ├── connection.rs # Connection state │ │ └── error.rs # Error types │ ├── handlers/ │ │ ├── mod.rs │ │ ├── auth.rs # Login, registration, OAuth │ │ ├── device.rs # Device registration endpoints │ │ ├── session.rs # Session CRUD │ │ ├── connection.rs # Connection mgmt │ │ ├── signaling.rs # WebRTC signaling │ │ ├── heartbeat.rs # Keep-alive │ │ └── logs.rs # Logging endpoints │ ├── middleware/ │ │ ├── mod.rs │ │ ├── auth.rs # JWT validation │ │ ├── rate_limit.rs # Rate limiting │ │ ├── cors.rs # CORS handling │ │ └── logging.rs # Request logging │ ├── services/ │ │ ├── mod.rs │ │ ├── user_service.rs # User business logic │ │ ├── device_service.rs # Device management │ │ ├── session_service.rs # Session creation/mgmt │ │ ├── auth_service.rs # JWT, password hashing │ │ ├── turn_service.rs # TURN/STUN config │ │ └── notification.rs # Real-time updates │ ├── db/ │ │ ├── mod.rs │ │ ├── migrations/ # SQLx migrations │ │ │ └── *.sql │ │ ├── queries.rs # Prepared queries │ │ └── connection.rs # Database utilities │ ├── websocket/ │ │ ├── mod.rs │ │ ├── handler.rs # WebSocket connection │ │ ├── messages.rs # Message types │ │ └── signaling.rs # WebRTC signaling logic │ ├── cache/ │ │ ├── mod.rs │ │ ├── redis_client.rs # Redis operations │ │ └── session_cache.rs # Session caching │ ├── utils/ │ │ ├── mod.rs │ │ ├── jwt.rs # JWT token creation │ │ ├── password.rs # Password hashing │ │ ├── errors.rs # Error handling │ │ └── logging.rs # Logging setup │ └── security/ │ ├── mod.rs │ ├── encryption.rs # Data encryption │ ├── validation.rs # Input validation │ └── sanitization.rs # XSS/injection prevention ├── Cargo.toml # Dependencies ├── Cargo.lock # Lock file ├── .env.example # Environment template └── tests/ ├── integration/ # Integration tests └── unit/ # Unit tests

Desktop Agent Structure (Rust)

desktop-agent/ ├── src/ │ ├── main.rs # Entry point │ ├── config/ │ │ ├── mod.rs │ │ ├── settings.rs # Configuration management │ │ ├── env.rs # Environment variables │ │ └── defaults.rs # Default settings │ ├── capture/ │ │ ├── mod.rs │ │ ├── dxgi.rs # DXGI screen capture │ │ ├── graphics.rs # Graphics Capture fallback │ │ ├── frame.rs # Frame data structure │ │ └── fps_controller.rs # FPS adjustment logic │ ├── encoder/ │ │ ├── mod.rs │ │ ├── h264.rs # H.264 codec │ │ ├── hardware/ │ │ │ ├── mod.rs │ │ │ ├── nvenc.rs # NVIDIA NVENC │ │ │ ├── qsv.rs # Intel Quick Sync │ │ │ └── amf.rs # AMD AMF │ │ └── software.rs # Software fallback │ ├── input/ │ │ ├── mod.rs │ │ ├── mouse.rs # Mouse control │ │ ├── keyboard.rs # Keyboard input │ │ ├── clipboard.rs # Clipboard sync │ │ └── events.rs # Input event types │ ├── network/ │ │ ├── mod.rs │ │ ├── webrtc.rs # WebRTC peer │ │ ├── signaling.rs # Signaling client │ │ ├── datachannel.rs # Data channel handling │ │ └── transport.rs # Transport layer │ ├── server/ │ │ ├── mod.rs │ │ ├── api.rs # Local HTTP API (optional) │ │ ├── handlers.rs # API handlers │ │ └── middleware.rs # Request middleware │ ├── system/ │ │ ├── mod.rs │ │ ├── monitor.rs # System monitoring │ │ ├── hardware.rs # Hardware detection │ │ ├── performance.rs # Performance metrics │ │ └── audio.rs # Audio detection (future) │ ├── auth/ │ │ ├── mod.rs │ │ ├── device_cert.rs # Device certificate │ │ └── token.rs # Token management │ ├── logging/ │ │ ├── mod.rs │ │ ├── file.rs # File logging │ │ ├── remote.rs # Remote logging │ │ └── metrics.rs # Metrics collection │ ├── protocol/ │ │ ├── mod.rs │ │ ├── messages.rs # Protocol message types │ │ ├── codec.rs # Message encoding │ │ └── errors.rs # Protocol errors │ ├── state/ │ │ ├── mod.rs │ │ ├── connection.rs # Connection state │ │ ├── session.rs # Session state │ │ └── metrics.rs # Metrics state │ └── utils/ │ ├── mod.rs │ ├── errors.rs # Error types │ ├── logging.rs # Logging utilities │ └── helpers.rs # Helper functions ├── Cargo.toml ├── build.rs # Build script └── tests/ ├── integration/ └── unit/

Chrome Extension Structure (TypeScript + React)

chrome-extension/ ├── public/ │ ├── manifest.json # Manifest V3 config │ ├── icon-16.png │ ├── icon-48.png │ ├── icon-128.png │ ├── icon-256.png │ └── index.html # Root HTML ├── src/ │ ├── index.tsx # Entry point │ ├── components/ │ │ ├── Dashboard.tsx # Main dashboard │ │ ├── Login.tsx # Login form │ │ ├── SessionList.tsx # Active sessions │ │ ├── RemoteScreen.tsx # Video viewer │ │ ├── Controls.tsx # Connection controls │ │ ├── Settings.tsx # Settings panel │ │ ├── ProfileMenu.tsx # User profile │ │ └── ConnectionStatus.tsx # Status indicator │ ├── pages/ │ │ ├── Home.tsx │ │ ├── Connect.tsx # Connection page │ │ ├── Session.tsx # Active session │ │ └── Settings.tsx │ ├── hooks/ │ │ ├── useAuth.ts # Authentication hook │ │ ├── useSession.ts # Session management │ │ ├── useWebRTC.ts # WebRTC connection │ │ ├── useInput.ts # Input handling │ │ └── useClipboard.ts # Clipboard operations │ ├── services/ │ │ ├── api.ts # Backend API client │ │ ├── auth.ts # Auth service │ │ ├── device.ts # Device service │ │ ├── session.ts # Session service │ │ ├── webrtc.ts # WebRTC setup │ │ └── signaling.ts # Signaling client │ ├── stores/ │ │ ├── authStore.ts # Zustand auth store │ │ ├── sessionStore.ts # Session state │ │ ├── uiStore.ts # UI state │ │ └── settingsStore.ts # Settings storage │ ├── types/ │ │ ├── index.ts # Type exports │ │ ├── auth.ts # Auth types │ │ ├── device.ts # Device types │ │ ├── session.ts # Session types │ │ ├── webrtc.ts # WebRTC types │ │ └── api.ts # API response types │ ├── utils/ │ │ ├── logger.ts # Logging │ │ ├── validators.ts # Input validation │ │ ├── formatters.ts # Data formatting │ │ └── helpers.ts # Utility functions │ ├── styles/ │ │ ├── globals.css # Global styles │ │ ├── tailwind.config.js # Tailwind config │ │ └── index.css # Component styles │ └── background/ │ ├── service-worker.ts # Background script │ └── offscreen.ts # Offscreen document (future) ├── vite.config.ts # Vite configuration ├── tsconfig.json ├── tailwind.config.js ├── package.json ├── package-lock.json └── tests/ └── **/*.test.ts

Technology Stack

Backend Stack

Desktop Agent Stack

Chrome Extension Stack

Networking & Communication

Protocol Stack

Transport Layer

WebRTC: Primary P2P connection for low-latency screen/input streaming

UDP preferred, TCP fallback for NAT traversal

Signaling

WebSocket: Backend-to-client signaling channel

Encrypted JSON messages for ICE candidates & SDP

Security

TLS 1.3: All backend-to-client communication

DTLS: WebRTC data channel encryption

SRTP: Audio/video stream encryption

Connection Establishment Flow

Client logs in via Chrome Extension → Backend validates JWT

Client requests session with host device ID

Backend verifies device authorization & sends TURN/STUN servers

Establishes WebSocket connection to backend for signaling
Backend notifies host device of pending connection
Exchange SDP & ICE candidates via WebSocket signaling
Host accepts connection, P2P link established
Host begins streaming encoded screen via WebRTC data channel
Security Architecture
Authentication
User Registration & Login
Email + password registration with email verification
Password hashing using Argon2 (memory-hard algorithm)
Google OAuth 2.0 integration for passwordless login
Session Management
JWT tokens with 24-hour expiration
Refresh tokens stored securely in Redis
Device fingerprinting for session binding
"Remember device" option with approval workflow
Encryption
In-Transit Encryption
TLS 1.3 for all HTTPS/WebSocket communication
DTLS 1.3 for WebRTC data channel encryption
SRTP (Secure Real-time Transport Protocol) for media streams
At-Rest Encryption
AES-256-GCM for sensitive database fields
Refresh tokens encrypted with rotating keys
Rate Limiting & DDoS Protection
Backend: 100 requests/minute per IP
Authentication endpoints: 5 attempts/5 minutes per email
WebSocket connections: 1 per user simultaneously
CloudFlare DDoS protection for production
Authorization & Access Control
Device ownership verification: Sessions only allowed with device owner
Permission scoping: Sessions require explicit device authorization
Session expiration: 12 hours maximum, configurable
Connection heartbeat: 30-second keep-alive, automatic cleanup on timeout
Data Protection
Screen data: Captured locally, never stored on backend
Input commands: Encrypted end-to-end via WebRTC
Clipboard sync: Verified ownership before sync

Audit logging: All connection events logged for security review

Screen Capture & Streaming Implementation

Capture Pipeline

Desktop Duplication API (DXGI) - Primary Method

DXGI provides GPU-accelerated screen capture with DirectX integration. It captures desktop frames directly from the GPU without copying to system RAM, resulting in minimal CPU overhead and maximum performance.

Graphics Capture API - Fallback

Windows.Graphics.Capture provides compatibility for older Windows versions where DXGI is unavailable.

Frame Processing

Frame Capture

30 FPS minimum, 60 FPS target

Delta Detection

Compare current frame with previous

Only encode changed regions for bandwidth savings

Adaptive FPS

Monitor network latency

Reduce FPS if latency exceeds 100ms

Increase FPS when latency drops below 50ms

Video Encoding

Codec Selection: H.264

H.264 provides universal browser support, hardware acceleration across most platforms, and proven compatibility.

Hardware Encoding Priority

NVIDIA NVENC (NvEnc)

Dedicated hardware encoder on NVIDIA GPUs

Intel Quick Sync

Integrated in Intel GPUs, power-efficient

AMD AMF

AMD GPU acceleration

Software Fallback

Use ffmpeg or libx264 for systems without hardware encoders

Network Transmission

WebRTC Data Channels: Stream H.264 frames to client

Adaptive Bitrate: Adjust encoding quality based on available bandwidth

Frame Ordering: Sequence numbers for frame verification

Packet Loss Recovery: Resend critical I-frames on loss

Client Rendering

H.264 Decoding: Browser native via HTMLVideoElement or WebCodecs API

Canvas Rendering: Display decoded frames in fullscreen canvas

Low-Latency Display: Minimize buffering for responsive feel

Troubleshooting Screen Sharing Issues

Database Design

Schema Overview

PostgreSQL is used as the primary persistent database. The schema is organized into logical domains with proper indexing for performance.

Core Tables

users

id (UUID, PK) • email (VARCHAR, UNIQUE) • password_hash (VARCHAR) • google_id (VARCHAR, nullable) • full_name (VARCHAR) • avatar_url (VARCHAR, nullable) • created_at (TIMESTAMP) • updated_at (TIMESTAMP) • last_login (TIMESTAMP, nullable) • is_active (BOOLEAN) • email_verified (BOOLEAN)

devices

id (UUID, PK) • user_id (UUID, FK users) • device_name (VARCHAR) • device_type (VARCHAR) • os_version (VARCHAR) • agent_version (VARCHAR) • device_token (VARCHAR, UNIQUE) • public_key (TEXT) • last_seen (TIMESTAMP) • ip_address (INET, nullable) • created_at (TIMESTAMP) • is_online (BOOLEAN) • allow_remote_access (BOOLEAN) • INDEX (user_id, is_online)

sessions

id (UUID, PK) • host_device_id (UUID, FK devices) • client_user_id (UUID, FK users) • session_token (VARCHAR, UNIQUE) • status (ENUM: active, inactive, expired) • started_at (TIMESTAMP) • ended_at (TIMESTAMP, nullable) • expires_at (TIMESTAMP) • connection_quality (VARCHAR) • bytes_sent (BIGINT) • bytes_received (BIGINT) • INDEX (host_device_id, status) • INDEX (client_user_id)

connections

id (UUID, PK) • session_id (UUID, FK sessions) • connection_type (VARCHAR) • latency_ms (INTEGER) • packet_loss (FLOAT) • bandwidth_kbps (INTEGER) • created_at (TIMESTAMP) • disconnected_at (TIMESTAMP, nullable) • INDEX (session_id)

refresh_tokens

id (UUID, PK) • user_id (UUID, FK users) • token_hash (VARCHAR, UNIQUE) • device_fingerprint (VARCHAR) • expires_at (TIMESTAMP) • created_at (TIMESTAMP) • INDEX (user_id, expires_at)

audit_logs

id (UUID, PK) • user_id (UUID, nullable, FK users) • device_id (UUID, nullable, FK devices) • session_id (UUID, nullable, FK sessions) • event_type (VARCHAR) • event_description (TEXT) • ip_address (INET) • user_agent (VARCHAR) • created_at (TIMESTAMP) • INDEX (user_id, created_at) • INDEX (device_id, created_at)

user_settings

id (UUID, PK) • user_id (UUID, FK users, UNIQUE) • session_timeout_minutes (INTEGER) • allow_remote_access_by_default (BOOLEAN) • notification_preferences (JSONB) • theme_preference (VARCHAR) • updated_at (TIMESTAMP)

API Specifications

Authentication Endpoints

POST /api/v1/auth/register

Register new user

Body: { email, password, full_name } → Returns: { user_id, access_token, refresh_token }

POST /api/v1/auth/login

Login with email/password

Body: { email, password, device_fingerprint } → Returns: { access_token, refresh_token, devices }

POST /api/v1/auth/google

Google OAuth callback

Body: { google_token } → Returns: { access_token, refresh_token }

POST /api/v1/auth/refresh

Refresh access token

Body: { refresh_token } → Returns: { access_token }

Device Endpoints

POST /api/v1/devices/register

Register new device (desktop agent)

Body: { device_name, os_version, public_key } → Returns: { device_id, device_token }

GET /api/v1/devices

List user devices

Headers: Authorization: Bearer {token} → Returns: [{ id, name, status, last_seen }]

GET /api/v1/devices/{device_id}

Get device details

Returns: Device object with full details

Session Endpoints

POST /api/v1/sessions/create

Initiate session with device

Body: { host_device_id } → Returns: { session_id, turn_servers, stun_servers }

POST /api/v1/sessions/{session_id}/accept

Host accepts connection request

Body: { device_token } → Returns: { status: 'accepted' }

DELETE /api/v1/sessions/{session_id}

End session

Returns: { status: 'disconnected' }

Integration Guide

For End Users

Desktop Agent Installation (Host)

Download ORBIT-Agent-Setup.exe from dashboard

Run installer (Admin privileges required)

Service auto-starts at login

Pair with account via device token

Chrome Extension Installation (Client)

Install from Chrome Web Store

Login with email or Google account

Select device from list and connect

For System Integrators

Self-hosted Backend: Deploy Docker containers to Kubernetes or VM

Custom Signaling: Implement alternative signaling protocol if needed

Enterprise Features: Add LDAP/AD authentication, IP restrictions, session recording

Branded Extension: White-label extension with custom domain

Performance Optimization

Latency Goals

Resource Utilization Targets

Idle State (No active session):

CPU: <2%

RAM: <100MB

Active Session (60 FPS screen share):

CPU: 5-15% (with hardware encoding)

RAM: 150-250MB

Bandwidth (LAN, 1080p 60FPS):

10-50 Mbps depending on screen activity

Backend Scalability

Horizontal scaling: Multiple backend instances behind load balancer

Database connection pooling: PgBouncer for PostgreSQL

Redis clustering: For session & cache layer

Estimated capacity: 1000+ concurrent sessions per backend instance

Deployment & DevOps

Docker Setup

Dockerfile.backend FROM rust:latest as builder WORKDIR /app COPY . . RUN cargo build --release FROM debian:bookworm-slim RUN apt-get update && apt-get install -y \ libpq5 \ ca-certificates && rm -rf /var/lib/apt/lists/* COPY --from=builder /app/target/release/orbit-backend /usr/local/bin/ EXPOSE 8080 CMD ["orbit-backend"]

Environment Configuration

.env.production RUST_LOG=info DATABASE_URL=postgresql://user:pass@db:5432/orbit REDIS_URL=redis://cache:6379 JWT_SECRET=<secure-random-key> TURN_SERVER=turn.example.com STUN_SERVER=stun.example.com TLS_CERT_PATH=/etc/certs/server.crt TLS_KEY_PATH=/etc/certs/server.key CORS_ALLOWED_ORIGINS=https://example.com

Kubernetes Deployment

Use Helm charts for consistent, reproducible deployments. Includes backend service, database, Redis, and monitoring.

CI/CD Pipeline (GitHub Actions)

Lint & format check on every commit

Run unit & integration tests

Build Docker image

Push to registry (ECR/Docker Hub)

Deploy to staging, run smoke tests, deploy to production

Conclusion

ORBIT v1.0 represents a comprehensive, production-grade remote desktop platform built with proven technologies and best practices. The architecture prioritizes speed, security, and reliability while remaining maintainable and extensible for future feature additions.

This specification provides developers with a complete roadmap from initial architecture through deployment, ensuring consistent implementation across all components.

Next Steps:

Set up development environment with Docker

Implement backend foundation (Auth, DB, API)

Build desktop agent with screen capture

Develop Chrome extension UI

Integrate WebRTC for P2P communication

Comprehensive security testing

Performance optimization & benchmarking

Beta release to selected users